

PRACTICAL WEEK – 6

TASK 1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
GNU nano 8.7.1 client.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char message[100];
    char response[100];
    int fd_write, fd_read;

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);
    message[strlen(message)] = '\0';

    fd_write = open(FIFO1, O_WRONLY);
    write(fd_write, message, strlen(message) + 1);
    close(fd_write);

    fd_read = open(FIFO2, O_RDONLY);
    read(fd_read, response, sizeof(response));
    close(fd_read);

    printf("Server response: %s\n", response);
    return 0;
}
```

```
shreyaj@LAPTOP-FT6VVM4D:~$ mkdir -p ~/practical-week-6/task1
cd ~/practical-week-6/task1
shreyaj@LAPTOP-FT6VVM4D:~/practical-week-6/task1$ pwd
/home/shreyaj/practical-week-6/task1
shreyaj@LAPTOP-FT6VVM4D:~/practical-week-6/task1$ nano server.c
shreyaj@LAPTOP-FT6VVM4D:~/practical-week-6/task1$ gcc server.c -o server
shreyaj@LAPTOP-FT6VVM4D:~/practical-week-6/task1$ ./server
Server started. Waiting for client...
Server received: hello server
Server response sent.
```

```
GNU nano 8.7.1 client.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char message[100];
    char response[100];
    int fd_write, fd_read;

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);
    message[strlen(message)] = '\0';

    fd_write = open(FIFO1, O_WRONLY);
    write(fd_write, message, strlen(message) + 1);
    close(fd_write);

    fd_read = open(FIFO2, O_RDONLY);
    read(fd_read, response, sizeof(response));
    close(fd_read);

    printf("Server response: %s\n", response);
    return 0;
}
```

```
shreyaj@LAPTOP-FT6VVM4D:~$ cd ~/practical-week-6/task1
shreyaj@LAPTOP-FT6VVM4D:~/practical-week-6/task1$ nano client.c
shreyaj@LAPTOP-FT6VVM4D:~/practical-week-6/task1$ gcc client.c -o client
shreyaj@LAPTOP-FT6VVM4D:~/practical-week-6/task1$ ./client
Enter message: hello server
Server response: Message received by server.
shreyaj@LAPTOP-FT6VVM4D:~/practical-week-6/task1$ █
```

TASK 2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
shreyaj@LAPTOP-FT6VVM4D:~$ mkdir -p ~/practical-week-6/task2
cd ~/practical-week-6/task2
shreyaj@LAPTOP-FT6VVM4D:~/practical-week-6/task2$ nano p-6.c
shreyaj@LAPTOP-FT6VVM4D:~/practical-week-6/task2$ gcc p-6.c -o p62
shreyaj@LAPTOP-FT6VVM4D:~/practical-week-6/task2$ ./p62
Signal handling program started.
PID: 1156
Waiting for signals...
^C
SIGINT received (Ctrl+C).

SIGUSR1 received.
```

```
GNU nano 8.7.1 p-6.c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void handle_sigint(int sig) {
    printf("\nSIGINT received (Ctrl+C).\n");
}

void handle_sigterm(int sig) {
    printf("\nSIGTERM received.\n");
}

void handle_sigusr1(int sig) {
    printf("\nSIGUSR1 received.\n");
}

int main() {
    signal(SIGINT, handle_sigint);
    signal(SIGTERM, handle_sigterm);
    signal(SIGUSR1, handle_sigusr1);

    printf("Signal handling program started.\n");
    printf("PID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1) {
        pause();
    }

    return 0;
}
```

```
shreyaj@LAPTOP-FT6VVM4D:~$ kill -SIGUSR1 1234
-bash: kill: (1234) - No such process
shreyaj@LAPTOP-FT6VVM4D:~$ kill -SIGUSR1 1156
```